

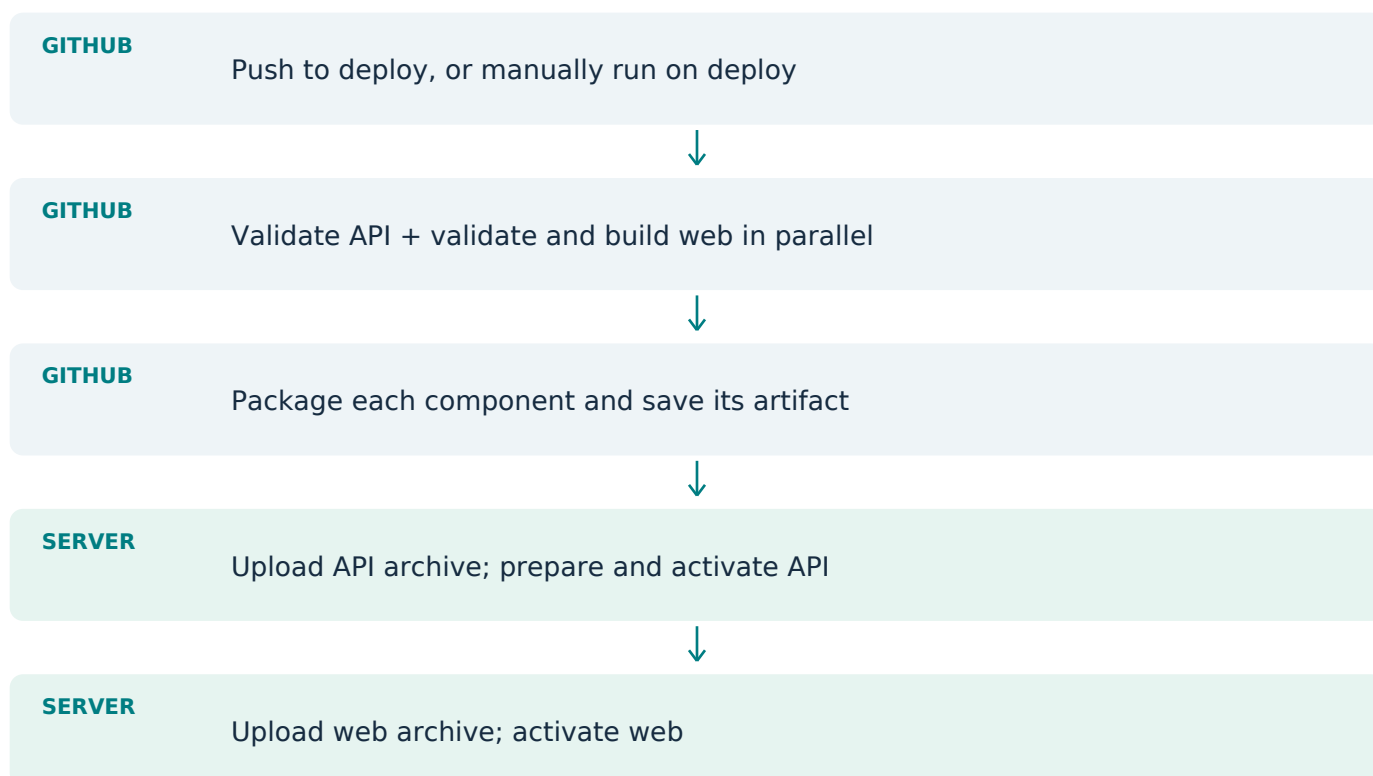
How deployment works

From a push on GitHub to a running API and website.

This guide explains your repository's routine deployment process. It includes the **complete workflow and both deployment scripts**, followed section by section with simple explanations. The server is assumed to be ready; one-time setup is outside this guide.

The main idea

.github/workflows/deploy.yml decides when and in what order tasks run. **deploy/package.sh** prepares the files to send. **deploy/release.sh** activates those files on your server.



Does everything run on every push? Each push starts the workflow. Both components are checked and packaged, even if only one changed. Later steps run only when their required earlier steps succeed. One-time setup scripts are never called by this workflow.

Contents and reading guide

Pages	What you will learn
1	The full deployment journey
2	How to read this document
3-9	Complete GitHub workflow, explained in order
10	Complete package.sh: create the release archives
11-14	Complete release.sh: prepare, activate and recover
15	What repeats, what persists and what happens on failure
16	Deployment tests, glossary and source references
17-21	Deployment Questions and Answers: ten polished answers from our discussion

How the code is presented

The complete source listings show every original line in order. The left gutter shows the repository line number. Long lines wrap visually; continuation rows have no line number. Indentation and code text are retained. The Q&A; appendix repeats selected snippets for explanation. The original three files are also embedded as PDF attachments for exact copying in a compatible PDF reader.

How to read GitHub YAML

jobs are groups of work. **steps** run in order inside a job. **uses** calls a reusable action; **run** executes commands. **needs** names jobs that must succeed first. **env** supplies settings to commands. **\${{ ... }}** reads a GitHub value, while **\$NAME** reads a shell variable.

Three different meanings of deploy

The **deploy branch** triggers this workflow. The **deploy/ directory** holds scripts. The **deploy Linux user** is the account that connects to the server. These share a name but have separate roles.

This is an explanation of the checked-in code, not a record of a live deployment. No server connection or deployment was performed to produce it.

1. Start the workflow

A push to the deploy branch starts the workflow. Manual runs are available too, but the job conditions allow work only on that branch.

.github/workflows/deploy.yml | lines 1-37

```
1 name: Deploy inventory
2
3 on:
4   push:
5     branches: [deploy]
6     workflow_dispatch:
7
8 permissions:
9   contents: read
10
11 concurrency:
12   group: inventory-production
13   cancel-in-progress: false
14
15 env:
16   RELEASE_ID: ${GITHUB_SHA}-${GITHUB_RUN_ID}-${GITHUB_RUN_ATTEMPT}
17
18 jobs:
19   validate-api:
20     if: github.ref == 'refs/heads/deploy'
21     runs-on: ubuntu-24.04
22     timeout-minutes: 20
23     services:
24       mysql:
25         image: mysql:8.0
26         env:
27           MYSQL_DATABASE: inventory_ci
28           MYSQL_USER: inventory_ci
29           MYSQL_PASSWORD: ci-only-password
30           MYSQL_ROOT_PASSWORD: ci-only-root-password
31         ports:
32           - 3306:3306
33         options: >-
34           --health-cmd="mysqladmin ping --silent"
35           --health-interval=10s
36           --health-timeout=5s
37           --health-retries=10
```

One release has one identity

RELEASE_ID combines the commit SHA, workflow run ID and retry attempt. API and web packages use the same identity. The concurrency group prevents overlapping active runs in this group; a newer push does not cancel the active run.

The API validation job uses Ubuntu and a temporary MySQL 8 service. The passwords here are CI test values, not production credentials. Each job has a 20-minute timeout. Repository token access is read-only.

2. Check the API

GitHub checks out the exact triggering commit and installs PHP 8.3 with Composer. These steps run on the runner, not your production server.

`.github/workflows/deploy.yml` | lines 38-76

```
38     steps:
39       - uses: actions/checkout@v4
40         with:
41           ref: ${github.sha}
42           persist-credentials: false
43       - uses: shivammathur/setup-php@v2
44         with:
45           php-version: '8.3'
46           extensions: bcmath, ctype, curl, dom, fileinfo, gd, intl, mbstring, pdo_mysql, pdo_sqlite, sqlite3, tokenizer, xml, zip
47           tools: composer:v2
48           coverage: none
49       - name: Test deployment scripts
50         run: python3 -m unittest discover -s deploy/tests
51       - name: Install API test dependencies
52         working-directory: inventory-api
53         run: composer install --prefer-dist --no-interaction --no-progress
54       - name: Test API
55         working-directory: inventory-api
56         run: |
57           cp .env.example .env
58           php artisan key:generate --force
59           php artisan test
60       - name: Check MySQL migrations
61         working-directory: inventory-api
62         env:
63           APP_ENV: testing
64           DB_CONNECTION: mysql
65           DB_HOST: 127.0.0.1
66           DB_PORT: '3306'
67           DB_DATABASE: inventory_ci
68           DB_USERNAME: inventory_ci
69           DB_PASSWORD: ci-only-password
70           DB_URL: ''
71           CACHE_STORE: array
72           SESSION_DRIVER: array
73           QUEUE_CONNECTION: sync
74         run: |
75           php artisan migrate --force
76           php artisan migrate:status
```

Three checks: test the deployment scripts, run the Laravel tests, then apply migrations to the temporary CI MySQL database. The `.env` and application key created here belong to the test workspace. Production data is not used for these checks.

3. Package API; start web checks

.github/workflows/deploy.yml | lines 77-110

```
77     - name: Install production API dependencies
78       working-directory: inventory-api
79       run: composer install --no-dev --prefer-dist --no-interaction --no-progress --
optimize-autoloader
80     - name: Package API
81       run: bash deploy/package.sh api "$RELEASE_ID" "$RUNNER_TEMP/packages"
82     - uses: actions/upload-artifact@v4
83       with:
84         name: api-${{ env.RELEASE_ID }}
85         path: |
86           ${ runner.temp }/packages/api-${{ env.RELEASE_ID }}.tar.gz
87           ${ runner.temp }/packages/api-${{ env.RELEASE_ID }}.tar.gz.sha256
88         if-no-files-found: error
89         retention-days: 7
90
91   validate-web:
92     if: github.ref == 'refs/heads/deploy'
93     runs-on: ubuntu-24.04
94     timeout-minutes: 20
95     steps:
96     - uses: actions/checkout@v4
97       with:
98         ref: ${ github.sha }
99         persist-credentials: false
100    - uses: actions/setup-node@v4
101      with:
102        node-version: '24'
103        cache: npm
104        cache-dependency-path: web/package-lock.json
105    - name: Install web dependencies
106      working-directory: web
107      run: npm ci --no-audit --no-fund
108    - name: Check web types
109      working-directory: web
110      run: npm run typecheck
```

Save a ready-to-deploy API

Composer installs production dependencies without development packages. package.sh then creates the API archive and checksum. upload-artifact stores both in GitHub for the deployment job to download. These GitHub artifacts are retained for seven days; that is not a server release cleanup policy.

Web validation is a separate job

The web job checks out the same commit, installs Node.js 24 and uses npm ci to install dependencies from the lockfile. Type checking catches TypeScript errors. It can run in parallel with API validation because neither validation job has a needs dependency.

4. Build web; gate deployment

.github/workflows/deploy.yml | lines 111-146

```
111     - name: Test web
112       working-directory: web
113       run: |
114         shopt -s globstar
115         node --test src/**/*.test.mjs
116     - name: Build web
117       working-directory: web
118       env:
119         NEXT_PUBLIC_API_URL: /api
120         NEXT_TELEMETRY_DISABLED: '1'
121       run: npm run build
122     - name: Package web
123       run: bash deploy/package.sh web "$RELEASE_ID" "$RUNNER_TEMP/packages"
124     - uses: actions/upload-artifact@v4
125       with:
126         name: web-`${{ env.RELEASE_ID }}`
127         path: |
128           `${{ runner.temp }}/packages/web-`${{ env.RELEASE_ID }}`.tar.gz
129           `${{ runner.temp }}/packages/web-`${{ env.RELEASE_ID }}`.tar.gz.sha256
130         if-no-files-found: error
131         retention-days: 7
132
133   deploy-api:
134     if: github.ref == 'refs/heads/deploy'
135     needs: [validate-api, validate-web]
136     runs-on: ubuntu-24.04
137     timeout-minutes: 20
138     steps:
139       - uses: actions/checkout@v4
140       with:
141         ref: `${{ github.sha }}`
142         persist-credentials: false
143       - uses: actions/download-artifact@v4
144       with:
145         name: api-`${{ env.RELEASE_ID }}`
146         path: `${{ runner.temp }}/packages
```

Build once, send the result

The web tests run before the Next.js production build. `NEXT_PUBLIC_API_URL=/api` makes the frontend use the `/api` path. `package.sh` packages the built web application; the server does not run `npm install` or `npm run build` during this release process.

Both sides must pass

`deploy-api` waits for both `validate-api` and `validate-web` to succeed. It then downloads the API archive from this workflow run. If either validation job fails, production deployment does not begin.

5. Prepare the SSH connection

.github/workflows/deploy.yml | lines 147-174

```

147     - name: Deploy api as deploy user
148       env:
149         DEPLOY_HOST: ${ secrets.DEPLOY_HOST }
150         DEPLOY_PORT: ${ secrets.DEPLOY_PORT }
151         DEPLOY_SSH_KEY: ${ secrets.DEPLOY_SSH_KEY }
152         DEPLOY_KNOWN_HOSTS: ${ secrets.DEPLOY_KNOWN_HOSTS }
153       shell: bash
154       run: |
155         set -euo pipefail
156         port=${DEPLOY_PORT:-22}
157         [[ "$DEPLOY_HOST" =~ ^[a-zA-Z0-9][a-zA-Z0-9.-]*$ ]] || { echo "Invalid DEPLOY_HOST"; exit 1; }
158         [[ "$port" =~ ^[0-9]{1,5}$ ]] || { echo "Invalid DEPLOY_PORT"; exit 1; }
159         (( 10#$port >= 1 && 10#$port <= 65535 )) || { echo "Invalid DEPLOY_PORT"; exit 1; }
160         [[ -n "$DEPLOY_SSH_KEY" && -n "$DEPLOY_KNOWN_HOSTS" ]] || { echo "Missing SSH configuration"; exit 1; }
161         ssh_dir="$(mktemp -d "$RUNNER_TEMP/inventory-ssh.XXXXXX")"
162         trap 'rm -rf "$ssh_dir"' EXIT
163         umask 077
164         printf '%s\n' "$DEPLOY_SSH_KEY" > "$ssh_dir/key"
165         printf '%s\n' "$DEPLOY_KNOWN_HOSTS" > "$ssh_dir/known_hosts"
166         unset DEPLOY_SSH_KEY DEPLOY_KNOWN_HOSTS
167         ssh_options=(
168           -i "$ssh_dir/key"
169           -o BatchMode=yes
170           -o IdentitiesOnly=yes
171           -o StrictHostKeyChecking=yes
172           -o "UserKnownHostsFile=$ssh_dir/known_hosts"
173           -o ConnectTimeout=15
174         )

```

What these four secrets mean

DEPLOY_HOST identifies the server. **DEPLOY_PORT** selects the SSH port, defaulting to 22.

DEPLOY_SSH_KEY authenticates the connection. **DEPLOY_KNOWN_HOSTS** supplies the expected server host key. This code references secret names; it does not contain their values.

The shell checks the inputs, writes SSH material to a private temporary directory and removes it when the shell exits. Strict host checking verifies the server against the supplied known-hosts file. BatchMode avoids an interactive password prompt.

6. Send API; queue web release

.github/workflows/deploy.yml | lines 175-204

```
175         cd "$RUNNER_TEMP/packages"
176         sha256sum --check "api-$RELEASE_ID.tar.gz.sha256"
177         scp "${ssh_options[@]}" -P "$port" \
178             "api-$RELEASE_ID.tar.gz" "api-$RELEASE_ID.tar.gz.sha256" \
179             "deploy@$DEPLOY_HOST:/var/www/inventory-deploy/incoming/"
180         ssh "${ssh_options[@]}" -p "$port" "deploy@$DEPLOY_HOST" \
181             "bash -s -- api $RELEASE_ID" < "$GITHUB_WORKSPACE/deploy/release.sh"
182
183     deploy-web:
184       if: github.ref == 'refs/heads/deploy'
185       needs: [deploy-api, validate-web]
186       runs-on: ubuntu-24.04
187       timeout-minutes: 20
188       steps:
189         - uses: actions/checkout@v4
190           with:
191             ref: ${ github.sha }
192             persist-credentials: false
193         - uses: actions/download-artifact@v4
194           with:
195             name: web-${ env.RELEASE_ID }
196             path: ${ runner.temp }/packages
197         - name: Deploy web as deploy user
198           env:
199             DEPLOY_HOST: ${ secrets.DEPLOY_HOST }
200             DEPLOY_PORT: ${ secrets.DEPLOY_PORT }
201             DEPLOY_SSH_KEY: ${ secrets.DEPLOY_SSH_KEY }
202             DEPLOY_KNOWN_HOSTS: ${ secrets.DEPLOY_KNOWN_HOSTS }
203           shell: bash
204           run: |
```

The exact connection to deploy/release.sh

sha256sum checks the API archive. scp copies the archive and checksum to the server's incoming directory. ssh then sends the local release.sh text into a remote Bash process, passing api and the release ID. The script runs on the server as the deploy user; a full repository checkout on the server is not needed.

Web waits for successful API activation

deploy-web depends on deploy-api and validate-web. It downloads the already-built web artifact, then prepares the same SSH settings. An API deployment failure prevents this web deployment job from running.

7. Send and activate web

.github/workflows/deploy.yml | lines 205-232

```

205         set -euo pipefail
206         port="${DEPLOY_PORT:-22}"
207         [[ "$DEPLOY_HOST" =~ ^[a-zA-Z0-9][a-zA-Z0-9.-]*$ ]] || { echo "Invalid DEPLOY_HOST"; exit 1; }
208         [[ "$port" =~ ^[0-9]{1,5}$ ]] || { echo "Invalid DEPLOY_PORT"; exit 1; }
209         (( 10#$port >= 1 && 10#$port <= 65535 )) || { echo "Invalid DEPLOY_PORT"; exit 1; }
210         [[ -n "$DEPLOY_SSH_KEY" && -n "$DEPLOY_KNOWN_HOSTS" ]] || { echo "Missing SSH configuration"; exit 1; }
211         ssh_dir="$(mktemp -d "$RUNNER_TEMP/inventory-ssh.XXXXXX")"
212         trap 'rm -rf "$ssh_dir"' EXIT
213         umask 077
214         printf '%s\n' "$DEPLOY_SSH_KEY" > "$ssh_dir/key"
215         printf '%s\n' "$DEPLOY_KNOWN_HOSTS" > "$ssh_dir/known_hosts"
216         unset DEPLOY_SSH_KEY DEPLOY_KNOWN_HOSTS
217         ssh_options=(
218             -i "$ssh_dir/key"
219             -o BatchMode=yes
220             -o IdentitiesOnly=yes
221             -o StrictHostKeyChecking=yes
222             -o "UserKnownHostsFile=$ssh_dir/known_hosts"
223             -o ConnectTimeout=15
224         )
225         cd "$RUNNER_TEMP/packages"
226         sha256sum --check "web-$RELEASE_ID.tar.gz.sha256"
227         scp "${ssh_options[@]}" -P "$port" \
228             "web-$RELEASE_ID.tar.gz" "web-$RELEASE_ID.tar.gz.sha256" \
229             "deploy@$DEPLOY_HOST:/var/www/inventory-deploy/incoming/"
230         ssh "${ssh_options[@]}" -p "$port" "deploy@$DEPLOY_HOST" \
231             "bash -s -- web $RELEASE_ID" < "$GITHUB_WORKSPACE/deploy/release.sh"
232

```

Same transport, different component

This repeats the temporary SSH setup, verifies the web checksum and uploads the web archive. The remote command passes web to release.sh, so the script follows its web branch instead of the API branch.

A successful workflow therefore calls package.sh twice on GitHub and release.sh twice on the server: once for API and once for web. The API activation completes before the web deployment begins.

8. Create the release archives

deploy/package.sh | lines 1-35

```

1  #!/usr/bin/env bash
2  set -Eeuo pipefail
3
4  component=${1:?Usage: package.sh api|web RELEASE_ID OUTPUT_DIR}
5  release=${2:?Release ID required}
6  output=${3:?Output directory required}
7  [[ $component == api || $component == web ]] || { echo 'Invalid component' >&2; exit 1
8  ; }
9  [[ $release =~ ^[a-zA-Z0-9][a-zA-Z0-9._-]{0,99}$ ]] || { echo 'Invalid release ID' >&2
10 ; exit 1; }
11 project=$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)
12 mkdir -p "$output"
13 output=$(cd "$output" && pwd)
14 archive="$component-$release.tar.gz"
15
16 if [[ $component == api ]]; then
17     test -f "$project/inventory-api/vendor/autoload.php"
18     tar -czf "$output/$archive" -C "$project/inventory-api" \
19         --exclude='./.env*' --exclude='./.git' --exclude='./node_modules' \
20         --exclude='./tests' --exclude='./storage' --exclude='./public/storage' \
21         --exclude='./bootstrap/cache/*.php' --exclude='./database/*.sqlite*' \
22         --exclude='./phpunit*' .
23 else
24     test -f "$project/web/.next/standalone/server.js"
25     staging=$(mktemp -d)
26     trap 'rm -rf "$staging"' EXIT
27     # Next's standalone server does not copy these assets itself.
28     cp -a "$project/web/.next/standalone/" "$staging/"
29     mkdir -p "$staging/.next"
30     cp -a "$project/web/.next/static" "$staging/.next/static"
31     if [[ -d $project/web/public ]]; then
32         cp -a "$project/web/public" "$staging/public"
33     fi
34     tar -czf "$output/$archive" -C "$staging" --exclude='./.env*' .
35 fi
36 (cd "$output" && sha256sum "$archive" > "$archive.sha256")
37 printf 'Packaged %s\n' "$output/$archive"

```

Inputs: component (api or web), release ID and output directory. The script validates these and names the archive component-release.tar.gz.

API: includes application code and installed vendor dependencies. It excludes .env files, storage, test files, SQLite databases and cached PHP configuration. **Web:** combines the standalone server with static assets and public files, excluding .env files.

Output: an archive plus a .sha256 checksum file. The checksum acts like a fingerprint, letting later steps detect changed or damaged archive bytes.

9. Select the target release

deploy/release.sh | lines 1-35

```

1  #!/usr/bin/env bash
2  set -Eeuo pipefail
3  umask 0027
4
5  # This script is streamed over SSH and always runs without root privileges.
6  [[ $(id -un) == deploy && $(id -u) != 0 ]] || { echo 'Run deployment as deploy, never
   root.' >&2; exit 1; }
7  mode=deploy
8  if [[ ${1:-} == rollback ]]; then mode=rollback; shift; fi
9  component=${1:?Usage: release.sh [rollback] api|web RELEASE_ID}
10 release=${2:?Release ID required}
11 [[ $component == api || $component == web ]] || { echo 'Invalid component' >&2; exit 1
   ; }
12 [[ $release =~ ^[a-zA-Z0-9][a-zA-Z0-9._-]{0,99}$ ]] || { echo 'Invalid release ID' >&2
   ; exit 1; }
13 root=${INVENTORY_DEPLOY_ROOT:-/var/www/inventory-deploy}
14 test -d "$root/incoming"
15 exec 9>"$root/.deploy.lock"
16 flock -w 300 9
17 target="$root/$component/releases/$release"
18 current="$root/$component/current"
19 previous=$(readlink -e "$current" || true)
20 switched=0
21
22 switch_to() {
23     ln -s "$1" "$current.next.$$"
24     mv -Tf "$current.next.$$" "$current"
25 }
26
27 restart_services() {
28     if [[ $component == api ]]; then
29         sudo -n /usr/bin/systemctl reload php8.3-fpm.service
30         sudo -n /usr/bin/systemctl restart inventory-queue.service
31         sudo -n /usr/bin/systemctl start inventory-scheduler.timer
32     else
33         sudo -n /usr/bin/systemctl restart inventory-web.service
34     fi
35 }

```

Entry checks: the script requires the non-root deploy user and validates the component and release ID. The server lock waits up to 300 seconds so cooperating release scripts cannot modify releases at the same time.

Folders: each release gets its own directory. current is a symbolic link - a pointer to the active directory. switch_to prepares a new link and replaces current in one filesystem operation.

Services: API activation reloads PHP-FPM, restarts the queue worker and starts the scheduler timer. Web activation restarts the Next.js service. These functions are defined here and called later; defining them does not execute their bodies.

10. Prepare failure recovery

deploy/release.sh | lines 36-63

```
36
37 failed() {
38     local status=$1
39     trap - ERR INT TERM
40     set +e
41     if [[ $switched == 1 ]]; then
42         if [[ -n $previous && -d $previous ]]; then
43             switch_to "$previous"
44             restart_services
45             echo "Restored previous $component code: $previous" >&2
46         else
47             unlink "$current"
48             if [[ $component == api ]]; then
49                 sudo -n /usr/bin/systemctl stop inventory-queue.service
50                 sudo -n /usr/bin/systemctl stop inventory-scheduler.timer
51             else
52                 sudo -n /usr/bin/systemctl stop inventory-web.service
53             fi
54             echo "First $component activation failed; no previous release exists." >&2
55         fi
56     fi
57     echo "Deployment failed. Database migrations were NOT reversed." >&2
58     exit "$status"
59 }
60 trap 'failed $?' ERR
61 trap 'failed 130' INT
62 trap 'failed 143' TERM
63
```

If activation has already started

The error handler attempts to switch the affected component back to its previous release and restart its services. If no previous release exists, it removes current and stops the web service, or the API queue and scheduler. Recovery commands are best-effort; this is not a guarantee that a broken service will recover.

If the switch has not happened

The previous current link stays in place. However, earlier preparation may already have changed the database. The handler does not reverse migrations. The ERR, INT and TERM traps handle command failures and supported interruption signals.

11. Prepare the new application

deploy/release.sh | lines 64-98

```

64 if [[ $mode == deploy ]]; then
65     test ! -e "$target"
66     archive="$component-$release.tar.gz"
67     (
68         cd "$root/incoming"
69         # Only accept a checksum for this exact artifact, never a supplied path.
70         read -r digest filename < "$archive.sha256"
71         [[ $digest =~ ^[a-f0-9]{64}$ && $filename == "$archive" ]]
72         printf '%s %s\n' "$digest" "$archive" | sha256sum --check --strict -
73     )
74     mkdir -p "$target"
75     tar -xzf "$root/incoming/$archive" -C "$target" --no-same-owner
76     if [[ $component == api ]]; then
77         test -f "$root/shared/api.env"
78         test -d "$root/shared/storage"
79         ln -s "$root/shared/api.env" "$target/.env"
80         ln -s "$root/shared/storage" "$target/storage"
81         mkdir -p "$target/bootstrap/cache"
82         chmod 2770 "$target/bootstrap/cache"
83         cd "$target"
84         composer check-platform-reqs --no-dev
85         php artisan migrate --force --no-interaction
86         php artisan storage:link --no-interaction
87         php artisan config:cache --no-interaction
88         php artisan route:cache --no-interaction
89         php artisan view:cache --no-interaction
90     else
91         test -f "$target/server.js"
92     fi
93     touch "$target/.ready"
94 else
95     # Rollback only switches already prepared code; it never runs migrations.
96     test -f "$target/.ready"
97 fi
98

```

Verify and extract: reject an existing target, verify the expected archive's checksum and unpack it into a new release directory. No switch has happened yet.

API preparation: link the existing production .env and shared storage, check platform requirements, run pending migrations, and prepare storage and caches. Already-applied migrations are not rerun. **Web:** check that server.js exists.

Ready marker: .ready records that preparation completed. Manual rollback selects an existing prepared release and skips extraction and migrations; normal workflow pushes use deploy mode.

12. Activate and check health

deploy/release.sh | lines 99-110

```
99  switched=1
100 switch_to "$target"
101 restart_services
102 if [[ $component == api ]]; then
103     curl --fail --silent --show-error --retry 15 --retry-delay 2 --retry-all-errors \
104         --connect-timeout 2 --max-time 5 http://127.0.0.1/up > /dev/null
105 else
106     curl --fail --silent --show-error --retry 15 --retry-delay 2 --retry-all-errors \
107         --connect-timeout 2 --max-time 5 http://127.0.0.1:3000/login > /dev/null
108 fi
109 trap - ERR INT TERM
110 printf 'Activated %s release %s as %s\n' "$component" "$release" "${id -un}"
```

Make the new code active

The script switches current to the prepared directory, then calls restart_services. It checks the API at `http://127.0.0.1/up` or the web login page at `http://127.0.0.1:3000/login`. Curl retries failures before giving up. These local checks confirm endpoint availability; they do not test every user journey or external network path.

A simple release example

Suppose `api/current` points to release A. A successful API deployment prepares release B and moves current to B. The same shared `.env` and storage are linked into B. If the health check fails, the handler attempts to point current back to A.

Server location	Purpose
incoming/	Uploaded archives and checksum files
api/releases/<release-id>/	Prepared API code for one release
web/releases/<release-id>/	Prepared web code for one release
api/current and web/current	Pointers to the active code
shared/api.env and shared/storage/	Configuration and persistent Laravel files

All paths in this table are under `/var/www/inventory-deploy` by default. The release script does not automatically delete old releases or incoming archives.

Every push: quick reference

Action	When and where
Validate API and web	Each eligible run, on GitHub; jobs can run in parallel.
Run package.sh	Once per component after its checks/build succeed, on GitHub.
Upload and run release.sh	API first, then web; only after prerequisites succeed.
Run production migrations	During API preparation; only pending migrations are applied.
Restart application services	During activation of the affected component.
Run one-time setup	Never called by this workflow.

What stays between releases

The production database remains in MySQL. The existing API .env and storage stay under shared/. New API releases link to them, so configuration and uploaded files are reused. Migrations can change database structure or data; persistence does not mean the database is untouched.

Failure	Result
Tests, type checks or build fail	Dependent deployment jobs do not proceed.
Checksum or preparation fails	The current link is not switched; any database changes already made are not undone.
Activation or health check fails	Attempt to restore the previous release of that component.
First activation fails	No previous code exists; remove current and stop the affected managed service(s).
Web fails after API succeeds	API stays on its newly deployed release; web recovery is separate.

Rollback is per component. This workflow does not provide an automatic, combined API + web + database rollback.

Tests, terms and source files

Why the workflow tests deploy/

The API validation job runs `python3 -m unittest discover -s deploy/tests`. The tests simulate release commands with temporary files and stand-in services. They check shared-data preservation, migration and checksum failures, health-check recovery, first-release failure, manual rollback, input validation and exclusion of sensitive/mutable files from API packages. They do not deploy to production or prove the real server is correctly configured.

Term	Plain-English meaning
Runner	The GitHub machine executing a job.
Artifact	Files saved by one job for another job to download.
Archive (.tar.gz)	Many files packed into one compressed file.
Checksum (.sha256)	A fingerprint used to check archive integrity.
SSH / SCP	A remote connection / file transfer over SSH.
Release	A prepared version of one application component.
Symbolic link	A filesystem pointer; current chooses the active release.
Migration	A versioned database change run by Laravel.
Health check	A request to see whether an application endpoint responds successfully.

Source snapshot

Full code reproduced from **`.github/workflows/deploy.yml`** (232 lines), **`deploy/package.sh`** (35 lines), and **`deploy/release.sh`** (110 lines). Test explanation is based on **`deploy/tests/test_deployment.py`**.

Repository: `inventory-basic-starter`. Commit at generation: `006b945`. Code reflects the working-tree files read for this document. One-time setup scripts and instructions are intentionally omitted.

Deployment Questions and Answers

Your deployment questions, polished into a practical reference. Repeated questions are combined; answers describe the current workflow unless a future option is explicitly stated.

Q1. How are `.github/` and `deploy/` connected?

`.github/workflows/deploy.yml` controls the sequence. It tells GitHub when to run checks, package the application and start a deployment. The **`deploy/` directory supplies the scripts** used to perform that work.

```
Push to deploy branch
-> GitHub workflow runs checks and builds web
-> package.sh creates API and web archives
-> workflow uploads the archives over SSH
-> release.sh activates API, then web, on the server
```

The `deploy` branch and `deploy/` directory are separate things. A folder named `deploy` does not run automatically; the workflow explicitly calls the scripts inside it.

Q2. What do the scripts do, and do they run on every push?

Script	Purpose and when it runs
<code>package.sh</code>	Runs on GitHub after checks pass. Packages API code and dependencies, or the built web server and assets. Produces an archive and checksum.
<code>release.sh</code>	Runs on the server after upload. Verifies and extracts the archive, prepares the app, switches the active release, restarts services and checks health.

On a successful workflow run, each script runs twice: once for API and once for web. Failed prerequisites prevent later steps. API release preparation also runs pending migrations and reuses the existing production `.env` and storage.

One-time setup is not repeated. `setup-server.sh` and `setup-ssh.sh` are not called by this workflow. There is no `setup.sh` in this repository. If activation fails, `release.sh` attempts to restore the previous component code; it does not reverse database migrations.

Branch checks and deployment order

Q3. What does the `github.ref` condition mean?

```
if: github.ref == 'refs/heads/deploy'
```

It means: **run this job only for the deploy branch**. `github.ref` is the full Git reference for the workflow run. `refs/heads/deploy` is the full branch reference, and `==` checks whether the two values match.

Selected branch	Reference	Job runs?
deploy	refs/heads/deploy	Yes, if its other prerequisites pass
main	refs/heads/main	No; skipped

The push trigger already selects the deploy branch. This additional job condition also protects manual runs: selecting main for a manual run does not bypass the branch restriction.

Q4. What does `group: inventory-production` do?

```
concurrency:  
  group: inventory-production  
  cancel-in-progress: false
```

It gives matching workflow runs one shared execution slot. In this repository, only one run in that group can execute at a time. Because this setting is at workflow level, it covers the checks and builds as well as deployment.

If run A is active when run B arrives, B waits. `cancel-in-progress: false` means B does not cancel A. With the current default queue behavior, if C arrives while B is still waiting, C replaces B; A continues running. This is not an unlimited queue.

`inventory-production` is simply the group label. It does not create a server or a GitHub environment.

Source: [GitHub Docs: control workflow concurrency](#).

Release IDs and test credentials

Q5. What is RELEASE_ID, and how is it used?

```
env:  
  RELEASE_ID: ${GITHUB_SHA}-${GITHUB_RUN_ID}-${GITHUB_RUN_ATTEMPT}
```

It identifies the code and workflow attempt being deployed. The commit SHA identifies the code; the run ID identifies this workflow run; the attempt number starts at 1 and increases when that run is retried.

Example (commit SHA shortened for readability):
abc123-987654-1

API archive: api-abc123-987654-1.tar.gz
Web archive: web-abc123-987654-1.tar.gz
API folder: api/releases/abc123-987654-1/

The actual ID uses the full commit SHA. API and web share the same ID. A rerun gets a new attempt number and therefore a separate release directory. Workflow-level env makes the value available to shell commands as \$RELEASE_ID.

Q6. Is it safe to put these MySQL credentials in the workflow?

```
MYSQL_DATABASE: inventory_ci  
MYSQL_USER: inventory_ci  
MYSQL_PASSWORD: ci-only-password  
MYSQL_ROOT_PASSWORD: ci-only-root-password
```

These fixed values are acceptable for a disposable CI test database containing no sensitive data. They must not be reused for production or other protected systems. The database name and user name are identifiers; the two password values grant access inside this test database service.

Here, GitHub starts a temporary MySQL container for the validation job. Laravel connects to it at 127.0.0.1 to check migrations. It is separate from the production database; MYSQL_ROOT_PASSWORD is not your server login password.

The production setup code defines separate database credentials with a generated password. That describes the repository configuration, not a live inspection of your server.

Where validation happens and why

Q7. Where does database migration validation happen?

On a temporary GitHub runner, before deployment. The validate-api job starts an Ubuntu runner and a MySQL service. The step named Check MySQL migrations connects Laravel to the inventory_ci database and runs:

```
php artisan migrate --force
php artisan migrate:status
```

The first command applies pending migrations; the second lists their status. This is a practice run on the CI database, not a command against production. The job's temporary MySQL container is removed when the job ends.

```
Push -> GitHub runner + temporary MySQL
      -> check migrations
      -> pass: deployment may continue if other checks pass
      -> fail: dependent deployment jobs stop
```

To see the result: open your repository on GitHub, choose **Actions**, open a **Deploy inventory** run, select **validate-api**, then expand **Check MySQL migrations**.

Your production database is changed later, when release.sh runs pending migrations during API deployment.

Q8. Why is migration validation necessary?

It catches database errors before they reach the live application. For example, a migration that creates a products table might contain invalid SQL or refer to a missing table. If that migration fails in CI, deployment stops before touching the production release.

It also exercises the migrations using MySQL, matching the database engine used by the server configuration. Without this check, a problem could first appear during production deployment.

It is a useful check, not a guarantee. The temporary database starts empty. It cannot reproduce every issue involving existing production rows, database size or server-specific settings. Production migrations still need to be designed carefully.

Running only what changed

Q9. If only inventory-api/ changes, does web still run?

Yes, in the current workflow. A push to deploy starts both validation jobs. There is no changed-file check selecting API or web jobs, so successful runs build, package and deploy both components even if only API files changed.

```
Only inventory-api/ changed
-> validate API + validate and build web
-> deploy API
-> deploy web
```

The same applies in the other direction: a web-only change still runs API validation and deployment. Later jobs proceed only when their required earlier jobs succeed.

Q10. Can unchanged components be skipped?

Yes, as a future workflow change. A preliminary job can detect which files changed, then job conditions can select the relevant component. The following is a possible policy, not the behavior currently implemented:

Change	Possible future behavior
Only inventory-api/	Validate and deploy API only
Only web/	Validate, build and deploy web only
Both components	Run both; deploy API before web
Shared release scripts or workflow	Run both
Only documentation or mobile	Skip API and web deployment
Manual run	Run both as an explicit default

Dependencies must change too. Currently deploy-api requires validate-web, and deploy-web requires deploy-api. GitHub normally skips a job when a required job is skipped. The revised conditions must allow deliberately skipped components while still blocking deployment after a required failure.

Source: [GitHub Docs: prerequisite jobs and skipping](#).

Status: selective deployment has not been implemented. Your workflow remains unchanged and continues to run both components.